Chapter 8 ▪ Generic Programming

Generic classes and methods have type parameters. This allows them to describe precisely how the types should be used in the implementation. Prior to generic classes, programmers had to use the `Object` class for writing code that works with multiple types. This was both cumbersome and unsafe.

With the introduction of generics, Java has an expressive type system that allows designers to describe in detail how types of variables and methods should vary. In straightforward situations, you will find it simple to implement generic code. In more advanced cases, it can get quite complex—for implementors. The goal is to provide classes and methods that other programmers can use without surprises.

The introduction of generics in Java 5 was a significant change in the Java programming language. A major design goal was to be backward compatible with earlier releases. As a result, Java generics have some uncomfortable limitations. You will learn about the benefits and challenges of generic programming in this chapter.

8.1. Type Parameters

Generic programming means writing code that can be reused for objects of many different types. For example, you don't want to program separate classes to collect `String` and `File` objects. And you don't have to—the single class `ArrayList` collects objects of any class. This is one example of generic programming.

Actually, Java had an `ArrayList` class before it had generic classes. Let us investigate how the mechanism for generic programming has evolved, and what that means for users and implementors.

8.1.1. The Advantage of Generic Programming

Before generic classes were added to Java, generic programming was achieved with *inheritance*. The `ArrayList` class simply maintained an array of `Object` references:

```
public class ArrayList { // before generic classes
    private Object[] elementData;
    . . .
    public Object get(int i) { . . . }
    public void add(Object o) { . . . }
}
```

This approach has two problems. A cast is necessary whenever you retrieve a value:

```
ArrayList files = new ArrayList();
. . .
String filename = (String) files.get(i);
```

Moreover, there is no error checking. You can add values of any class:

```
files.add(Path.of(". . ."));
files.add(new File(". . ."));
```

These calls compile and run without error. Elsewhere, casting the result of `files.get(i)` to a `String` will throw an exception.

Generics offer a better solution: *type parameters*. The `ArrayList` class now has a type parameter that indicates the element type:

```
var files = new ArrayList<String>();
```

This makes your code easier to read. You can tell right away that this particular array list contains `String` objects.



Note: If you declare a variable with an explicit type instead of `var`, you can omit the type parameter in the constructor by using the "diamond" syntax:

```
ArrayList<String> files = new ArrayList<>();
```

The omitted type is inferred from the type of the variable. (The nickname of the syntax comes from the empty `<>` that vaguely looks like a diamond.)

The compiler can make good use of the type information too. No cast is required for calling `get`. The compiler knows that the return type is `String`, not `Object`:

```
String filename = files.get(0);
```

The compiler also knows that the `add` method of an `ArrayList<String>` has a parameter of type `String`. That is a lot safer than having an `Object` parameter. Now the compiler can check that you don't insert objects of the wrong type. For example, the statement

```
files.add(new File(". . ."); // can only add String objects to an ArrayList<String>
```

will not compile. A compiler error is much better than a class cast exception at runtime.

This is the appeal of type parameters: They make your programs easier to read and safer.

8.1.2. Who Wants to Be a Generic Programmer?

It is easy to use a generic class such as `ArrayList`. Most Java programmers will simply use types such as `ArrayList<String>` as if they had been built into the language, just like `String[]` arrays. (Of course, array lists are better than arrays because they can expand automatically.)

However, it is not so easy to implement a generic class. The programmers who use your code will want to plug in all sorts of classes for your type parameters. They will expect everything to work without onerous restrictions and confusing error messages. Your job as a generic programmer, therefore, is to anticipate all the potential future uses of your class.

How hard can this get? Here is a typical issue that the designers of the standard class library had to grapple with. The `ArrayList` class has a method `addAll` to add all elements of another collection. A programmer may want to add all elements from an `ArrayList<Manager>` to an `ArrayList<Employee>`. But, of course, doing it the other way round should not be legal. How do you allow one call and disallow the other? The Java language designers invented an ingenious new concept, the *wildcard type*, to solve this problem. Wildcard types are rather abstract, but they allow a library builder to make methods as flexible as possible.

Generic programming falls into three skill levels. At a basic level, you just use generic classes—typically, collections such as `ArrayList`—without thinking how and why they work. Most application programmers will want to stay at that level until something goes wrong. You may, however, encounter a confusing error message when mixing different generic classes. At that point, you'll need to learn enough about Java generics to solve problems systematically rather than through random tinkering. Finally, of course, you may want to implement your own generic classes and methods.

In this chapter, I will show you everything you need to know to implement your own generic code. However, I expect that most readers will use this knowledge primarily for help with troubleshooting and to satisfy their curiosity about the inner workings of the generic classes that they use regularly.

8.1.3. Defining a Simple Generic Class

A *generic class* is a class with one or more type variables. In this chapter, I use a simple `Pair` class as the primary example. This class allows us to focus on generics without being distracted by data storage details. Here is the code for the generic `Pair` class:

```
public class Pair<T> {
    private T first;
    private T second;

    public Pair() { first = null; second = null; }
    public Pair(T first, T second) { this.first = first; this.second = second; }

    public T getFirst() { return first; }
    public T getSecond() { return second; }

    public void setFirst(T newValue) { first = newValue; }
    public void setSecond(T newValue) { second = newValue; }
}
```

Why not just use an immutable record? As you will soon see, there is a reason for making this class mutable.

The `Pair` class introduces a type variable `T`, enclosed in angle brackets `< >`, after the class name. A generic class can have more than one type variable. For example, I could have defined the `Pair` class with separate types for the first and second field:

```
public class Pair<T, U> { . . . }
```

The type variables are used throughout the class definition to specify method return types and the types of fields and local variables. For example:

```
private T first; // uses the type variable
```



Note: It is common practice to use uppercase letters for type variables, and to keep them short. The Java library uses the variable `E` for the element type of a collection, `K` and `V` for key and value types of a table, and `T` (and the neighboring letters `U` and `S`, if necessary) for "any type at all."

You *instantiate* the generic type by substituting types for the type variables, such as

```
Pair<String>
```

You can think of the result as an ordinary class with constructors

```
Pair<String>()
Pair<String>(String, String)
```

and methods

```
String getFirst()
String getSecond()
void setFirst(String)
void setSecond(String)
```

In other words, the generic class acts as a factory for ordinary classes.

The program in **Listing 8.1** puts the `Pair` class to work. The `minmax` method traverses an array and simultaneously computes the minimum and maximum values. It uses a `Pair` object to return both results.

Listing 8.1 v1ch08/PairDemo1.java

```
1  package v1ch08;
2
3  import com.horstmann.corejava.Pair;
4
5  class PairDemo1 {
6      void main() {
7          String[] words = { "Mary", "had", "a", "little", "lamb" };
8          Pair<String> mm = minmax(words);
9          IO.println("min = " + mm.getFirst());
10         IO.println("max = " + mm.getSecond());
11     }
12
13     /**
14      * Gets the minimum and maximum of an array of strings.
15      * @param a an array of strings
16      * @return a pair with the min and max values, or null if a is null or empty
17      */
18     Pair<String> minmax(String[] a) {
19         if (a == null || a.length == 0) return null;
20         String smallest = a[0];
21         String largest = a[0];
22         for (int i = 1; i < a.length; i++) {
23             if (smallest.compareTo(a[i]) > 0) smallest = a[i];
24             if (largest.compareTo(a[i]) < 0) largest = a[i];
25         }
26         return new Pair<>(smallest, largest);
27     }
28 }
```

8.1.4. Generic Methods

In the preceding section, you have seen how to define a generic class. You can also define a single method with type parameters.

```
<T> T getMiddle(T... a) {
    return a[a.length / 2];
}
```

This method is defined inside an ordinary class, not inside a generic class. However, it is a generic method, as you can see from the angle brackets and the type variable. Note that the type variables are inserted after the modifiers (`public static`, in our case) and before the return type.

You can define generic methods both inside ordinary classes and inside generic classes.

When you call a generic method, you do not normally provide any generic type parameter. The compiler has enough information to infer the method that you want. In the following expression, the type of the arguments implies that `T` must be `String`:

```
getMiddle("John", "Q.", "Public")
```

In the rare case that you must specify a type, place it, enclosed in angle brackets, directly before the method name:

```
Collections.<String>emptySet()
```



Note: Perhaps you find that placing the type before the method looks weird. But there is a good reason.

In C++, you place the type arguments after the method name. That can lead to nasty parsing ambiguities. For example, `g(f<a,b>(c))` can mean "call `g` with the result of `f<a,b>(c)`," or "call `g` with the two `boolean` values `f<a` and `b>(c)`."

In Java, the type arguments must follow a dot and precede the method name. With an instance method, for example in a compact source file, it looks like this:

```
this.<String>getMiddle("John", "Q.", "Public")
```

In almost all cases, type inference for generic methods works smoothly. Occasionally, the compiler gets it wrong, and you'll need to decipher an error message. Consider this example:

```
double middle = getMiddle(3.14, 1729, 0);
```

There is a cryptic error message that varies from one compiler version to another, somewhat like this one:

```
incompatible types: inferred type does not conform to upper bound(s)
    inferred: java.lang.Number&java.lang.Comparable<? extends
java.lang.Number&java.lang.Comparable<?>&java.lang.constant.Constable
```

```
&java.lang.constant.ConstantDesc>&java.lang.constant.Constable
&java.lang.constant.ConstantDesc
    upper bound(s): java.lang.Double,java.lang.Object
```

The compiler is struggling to find a common type `T` for the `double` and `int` arguments. It actually found two: `Number` and the `Comparable` interface, which is itself a generic type. In this case, the remedy is to write all arguments as `double` values.



Tip: Peter von der Ahé recommends this trick if you want to see which type the compiler infers for a generic method call: Purposefully introduce an error and study the resulting error message. For example, consider the call `getMiddle("Hello", 0, null)`. Assign the result to a `LocalDate`, which can't possibly be right. You will get an error report:

```
inferred: java.lang.Object&java.io.Serializable&java.lang.Comparable<? extends
    java.lang.Object&java.io.Serializable&java.lang.Comparable<?>
    &java.lang.constant.Constable&java.lang.constant.ConstantDesc>
&java.lang.constant.Constable&java.lang.constant.ConstantDesc
```

In plain English, you can assign the result to a variable of any of these types: `Object`, `Serializable`, `Comparable`, `Constable`, or `ConstantDesc`.



Caution: Inferring large numbers of types can be slow. Suppose you call

```
List.of(
    new Pair<>("John", "Smith"),
    new Pair<>("Betty", "Jones"),
    . . .
)
```

with a few hundred pairs. It takes the compiler a surprisingly long time to figure out that all the diamonds should be `<String>`. If you help the compiler out, it is much faster:

```
List.<Pair<String>>of(
    new Pair<>("John", "Smith"),
    new Pair<>("Betty", "Jones"),
    . . .
)
```

8.1.5. Bounds for Type Variables

Sometimes, a class or a method needs to place restrictions on type variables. Here is a typical example. We want to compute the smallest element of an array of any type:

```
<T> T min(T[] a) { // almost correct
    if (a == null || a.length == 0) return null;
    T smallest = a[0];
    for (int i = 1; i < a.length; i++) {
        if (smallest.compareTo(a[i]) > 0) smallest = a[i];
    }
    return smallest;
}
```

But there is a problem. Look inside the code of the `min` method. The variable `smallest` has type `T`, which means it could be an object of an arbitrary class. How do we know that the class to which `T` belongs has a `compareTo` method?

The solution is to restrict `T` to a class that implements the `Comparable` interface—a standard interface with a single method, `compareTo`. You can achieve this by giving a *bound* for the type variable `T`:

```
<T extends Comparable> T min(T[] a) . . .
```

Actually, the `Comparable` interface is itself a generic type. For now, ignore that complexity and the warnings that the compiler generates. **Section 8.4** discusses how to properly use type parameters with the `Comparable` interface.

Now, the generic `minmax` method can only be called with arrays of classes that implement the `Comparable` interface, such as `String`, `LocalDate`, and so on. Calling `minmax` with a `Rectangle` array is a compile-time error because the `Rectangle` class does not implement `Comparable`.

You may wonder about the `extends` keyword. Why not use the `implements` keyword in this situation? After all, `Comparable` is an interface. The notation

```
<T extends BoundingType>
```

expresses that `T` should be a *subtype* of the bounding type. Both `T` and the bounding type can be either a class or an interface. The `extends` keyword was chosen because it is a reasonable approximation of the subtype concept, and the Java designers did not want to add a new keyword to the language.

A type variable can have multiple bounds. For example:

```
T extends Comparable & Serializable
```

The bounding types are separated by ampersands (`&`) because commas are used to separate type variables.

As with Java inheritance, you can have as many interface supertypes as you like, but at most one of the bounds can be a class. If you have a class as a bound, it

must be the first one in the bounds list.

In the next sample program (**Listing 8.2**), we rewrite the `minmax` method to be generic. The method computes the minimum and maximum of a generic array, returning a `Pair<T>`.

Listing 8.2 v1ch08/PairDemo2.java

```
1  package v1ch08;
2
3  import module java.base;
4  import com.horstmann.corejava.Pair;
5
6  class PairDemo2 {
7      void main() {
8          LocalDate[] birthdays = {
9                  LocalDate.of(1906, 12, 9), // G. Hopper
10                 LocalDate.of(1815, 12, 10), // A. Lovelace
11                 LocalDate.of(1903, 12, 3), // J. von Neumann
12                 LocalDate.of(1910, 6, 22), // K. Zuse
13         };
14         Pair<LocalDate> mm = minmax(birthdays);
15         IO.println("min = " + mm.getFirst());
16         IO.println("max = " + mm.getSecond());
17     }
18
19     /**
20      * Gets the minimum and maximum of an array of objects of type T.
21      * @param a an array of objects of type T
22      * @return a pair with the min and max values, or null if a is null or empty
23      */
24     <T extends Comparable> Pair<T> minmax(T[] a) {
25         if (a == null || a.length == 0) return null;
26         T smallest = a[0];
27         T largest = a[0];
28         for (int i = 1; i < a.length; i++) {
29             if (smallest.compareTo(a[i]) > 0) smallest = a[i];
30             if (largest.compareTo(a[i]) < 0) largest = a[i];
31         }
32         return new Pair<>(smallest, largest);
33     }
34 }
```

8.1.6. Generic Exceptions

A method can have a `throws` clause with a type variable. Consider this functional interface:

```
interface Task<T extends Throwable> {
    void run() throws T;
}
```

You can then propagate the generic exception in methods that use the functional interface:

```
<T extends Throwable> void repeat(Task<T> t, int n) throws T {
    for (int i = 0; i < n; i++) t.run();
}
```

For example, the `Thread.sleep` method is declared to throw the checked `InterruptedException`. Therefore, the call

```
repeat(() -> Thread.sleep(100), 10)
```

throws the same exception.



Caution: If the lambda expression throws both an `InterruptedException` and an `IOException`, then the inferred type for `T` is the least upper bound, `Exception`. We would like it to be `InterruptedException | IOException`, but that is not an actual Java type.

You cannot use a type variable in a `catch` clause. For example, the following method will not compile:

```
<T extends Throwable> void repeat(Task<T> t, int n) {
    try {
        for (int i = 0; i < n; i++) t.run();
    } catch (T e) { // ERROR
        e.printStackTrace();
    }
}
```

You can neither throw nor catch objects of a generic class. In fact, it is not even legal for a generic class to extend `Throwable`. For example, the following definition will not compile:

```
public class Problem<T> extends Exception { /* . . . */ }
    // ERROR--can't extend Throwable
```

8.2. Generic Code and the Virtual Machine

The virtual machine does not have objects of generic types—all objects belong to ordinary classes. An earlier version of the generics implementation was even able to compile a program that used generics into class files that executed on 1.0 virtual machines! In the following sections, you will see how the compiler “erases” type parameters, and what implication that process has for Java programmers.

8.2.1. Type Erasure

Whenever you define a generic type, a corresponding *raw* type is automatically provided. The name of the raw type is simply the name of the generic type, with the type parameters removed. The type variables are *erased* and replaced by their bounding types (or `Object` for variables without bounds).

For example, the raw type for `Pair<T>` looks like this:

```
public class Pair {
    private Object first;
    private Object second;

    public Pair(Object first, Object second) {
        this.first = first;
        this.second = second;
    }

    public Object getFirst() { return first; }
    public Object getSecond() { return second; }

    public void setFirst(Object newValue) { first = newValue; }
    public void setSecond(Object newValue) { second = newValue; }
}
```

Since `T` is an unbounded type variable, it is simply replaced by `Object`.

The result is an ordinary class, just as you might have implemented it before generics were added to Java.

Your programs may contain different kinds of `Pair`, such as `Pair<String>` or `Pair<LocalDate>`, but erasure turns them all into raw `Pair` types.



Note: Erasure is efficient, but it only works because all type parameters are reference types. In contrast, the C++ compiler must generate separate code for generic methods with different type parameters. C# takes a middle ground, providing one implementation for all reference types and separate ones for each *value type*. (Value types are primitive types and other types that can be stored directly, without needing a reference.)

There are plans to add value types to Java, and allow them as generic type parameters. The virtual machine will optimize the memory layout, placing values directly into fields and arrays when possible. Sometime in the future, you will be able to use an `ArrayList<int>` that is backed by an `int[]` array, rather than an array of references to `Integer` objects.

The raw type replaces type variables with the first bound, or `Object` if no bounds are given. For example, the type variable in the class `Pair<T>` has no explicit bounds, hence the raw type replaces `T` with `Object`. Suppose we declare a slightly different type:

```
public class Interval<T extends Comparable & Serializable> implements Serializable {
    private T lower;
    private T upper;
    . . .
    public Interval(T first, T second) {
        if (first.compareTo(second) <= 0) { lower = first; upper = second; }
        else { lower = second; upper = first; }
    }
}
```

The raw type `Interval` looks like this:

```
public class Interval implements Serializable {
    private Comparable lower;
    private Comparable upper;
    . . .
    public Interval(Comparable first, Comparable second) { . . . }
}
```



Note: You may wonder what happens if you switch the bounds: `class Interval<T extends Serializable & Comparable>`. In that case, the raw type replaces `T` with `Serializable`, and the compiler inserts casts to `Comparable` when necessary.

8.2.2. Translating Generic Expressions

When you program a call to a generic method, the compiler inserts casts when the return type has been erased. For example, consider the sequence of statements

```
Pair<Employee> buddies = . . .;
Employee buddy = buddies.getFirst();
```

The erasure of `getFirst` has return type `Object`. The compiler automatically inserts the cast to `Employee`. That is, the compiler translates the method call into two virtual machine instructions:

- A call to the raw method `Pair.getFirst`
- A cast of the returned `Object` to the type `Employee`

Casts are also inserted when you access a generic field. Suppose the `first` and `second` fields of the `Pair` class were public. (Not a good programming style, perhaps, but it is legal Java.) Then the expression

```
Employee buddy = buddies.first;
```

also has a cast inserted in the resulting bytecodes.

8.2.3. Translating Generic Methods

Type erasure also happens for generic methods. Programmers usually think of a generic method such as

```
<T extends Comparable> T min(T[] a)
```

as a whole family of methods, but after erasure, only a single method is left:

```
Comparable min(Comparable[] a)
```

Note that the type parameter `T` has been erased, leaving only its bounding type `Comparable`.

Erasure of methods brings up a couple of complexities. Consider this example:

```
class DateInterval extends Pair<LocalDate> {
    public void setSecond(LocalDate second) {
        if (second.compareTo(getFirst()) >= 0)
            super.setSecond(second);
    }
    . . .
}
```

A date interval is a pair of `LocalDate` objects, and we'll want to override the methods to ensure that the second value is never smaller than the first. This class is erased to

```
class DateInterval extends Pair { // after erasure
    public void setSecond(LocalDate second) { . . . }
    . . .
}
```

Perhaps surprisingly, there is another `setSecond` method, inherited from `Pair`, namely

```
public void setSecond(Object second)
```

This is clearly a different method because it has a parameter of a different type—`Object` instead of `LocalDate`. But it *shouldn't* be different. Consider this sequence of statements:

```
var interval = new DateInterval(. . .);
Pair<LocalDate> pair = interval; // OK--assignment to superclass
pair.setSecond(aDate);
```

Our expectation is that the call to `setSecond` is polymorphic and that the appropriate method is called. Since `pair` refers to a `DateInterval` object, that should be `DateInterval.setSecond`. The problem is that the type erasure interferes with polymorphism. To fix this problem, the compiler generates a *bridge method* in the `DateInterval` class:

```
public void setSecond(Object second) { setSecond((LocalDate) second); }
```

To see why this works, let us carefully follow the execution of the statement

```
pair.setSecond(aDate)
```

The variable `pair` has declared type `Pair<LocalDate>`, and that type only has a single method called `setSecond`, namely `setSecond(Object)`. The virtual machine calls that method on the object to which `pair` refers. That object is of type `DateInterval`. Therefore, the method `DateInterval.setSecond(Object)` is called. That method is the synthesized bridge method. It calls `DateInterval.setSecond(LocalDate)`, which is what we want.

Bridge methods can get even stranger. Suppose the `DateInterval` class also overrides the `getSecond` method:

```
class DateInterval extends Pair<LocalDate> {
    public LocalDate getSecond() { return (LocalDate) super.getSecond(); }
    . . .
}
```

In the `DateInterval` class, there are two `getSecond` methods:

```
LocalDate getSecond() // defined in DateInterval
Object getSecond() // overrides the method defined in Pair to call the first method
```

You could not write Java code like that; it would be illegal to have two methods with the same parameter types—here, with no parameters. However, in the virtual machine, the parameter types *and the return type* specify a method. Therefore, the compiler can produce bytecodes for two methods that differ only in their return type, and the virtual machine will handle this situation correctly.



Note: Bridge methods are not limited to generic types. You already saw in **Chapter 5** that it is legal for a method to specify a more restrictive return type when overriding another method. For example:

```
public class Employee implements Cloneable {
    public Employee clone() throws CloneNotSupportedException { . . . }
}
```

The `Object.clone` and `Employee.clone` methods are said to have *covariant return types*.

Actually, the `Employee` class has *two* clone methods:

```
Employee clone() // defined above
Object clone() // synthesized bridge method, overrides Object.clone
```

The synthesized bridge method calls the newly defined method.

In summary, you need to remember these facts about translation of Java generics:

- There are no generics in the virtual machine, only ordinary classes and methods.
- All type parameters are replaced by their bounds.
- Bridge methods are synthesized to preserve polymorphism.
- Casts are inserted as necessary to preserve type safety.

8.2.4. Calling Legacy Code

When Java generics were designed, a major goal was to allow interoperability between generics and legacy code. Let us look at a concrete example of such legacy. The Swing user interface toolkit provides a `JSlider` class whose "ticks" can be customized with labels that contain text or images. The labels are set with the call

```
void setLabelTable(Dictionary table)
```

The `Dictionary` class maps integers to labels. Before Java 5, that class was implemented as a map of `Object` instances. Java 5 made `Dictionary` into a generic class, but `JSlider` was never updated. At this point, `Dictionary` without type parameters is a raw type. This is where compatibility comes in.

When you populate the dictionary, you can use the generic type.

```
Dictionary<Integer, Component> labelTable = new Hashtable<>();
labelTable.put(0, new JLabel(new ImageIcon("nine.png")));
labelTable.put(20, new JLabel(new ImageIcon("ten.png")));
. . .
```

When you pass the `Dictionary<Integer, Component>` object to `setLabelTable`, the compiler issues a warning.

```
slider.setLabelTable(labelTable); // warning
```

After all, the compiler has no assurance about what the `setLabelTable` might do to the `Dictionary` object. That method might replace all the keys with

strings. That breaks the guarantee that the keys have type `Integer`, and future operations may cause class cast exceptions.

You should ponder it and ask what the `JSlider` is actually going to do with this `Dictionary` object. In our case, it is pretty clear that the `JSlider` only reads the information, so you can ignore the warning.

Now consider the opposite case, in which you get an object of a raw type from a legacy class. You can assign it to a variable whose type uses generics, but of course you will get a warning. For example:

```
Dictionary<Integer, Components> labelTable = slider.getLabelTable(); // warning
```

That's OK—review the warning and make sure that the label table really contains `Integer` and `Component` objects. Of course, there never is an absolute guarantee. A malicious coder might have installed a different `Dictionary` in the slider. But again, the situation is no worse than it was before generics. In the worst case, your program will throw an exception.

After you are done pondering the warning, you can use an *annotation* to make it disappear. You can annotate a local variable:

```
@SuppressWarnings("unchecked")
Dictionary<Integer, Components> labelTable = slider.getLabelTable(); // no warning
```

Or you can annotate an entire method, like this:

```
@SuppressWarnings("unchecked")
public void configureSlider() { . . . }
```

This annotation turns off checking for all code inside the method.

8.3. Inheritance Rules for Generic Types

When you work with generic classes, you need to learn a few rules about inheritance and subtypes. Let's start with a situation which many programmers find unintuitive. Consider a class and a subclass, such as `Employee` and `Manager`. Is `Pair<Manager>` a subtype of `Pair<Employee>`? Perhaps surprisingly, the answer is "no." For example, the following code will not compile:

```
Pair<Employee> buddies = new Pair<Manager>(ceo, cfo); // illegal
```

In general, there is *no* relationship between `Pair<S>` and `Pair<T>`, no matter how `S` and `T` are related (see **Figure 8.1**).

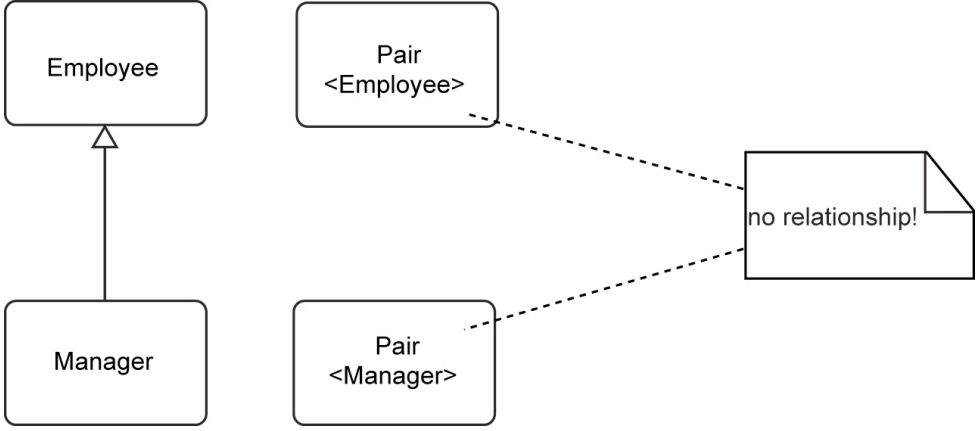


Figure 8.1: No inheritance relationship between `Pair` classes

This seems like a cruel restriction, but it is necessary for type safety. Suppose we were allowed to convert a `Pair<Manager>` to a `Pair<Employee>`. Consider this code:

```
var managerBuddies = new Pair<Manager>(ceo, cfo);
Pair<Employee> employeeBuddies = managerBuddies; // illegal, but suppose it wasn't
employeeBuddies.setFirst(lowlyEmployee);
```

Clearly, the last statement is legal. But `employeeBuddies` and `managerBuddies` refer to the *same object*. We now managed to pair up the CFO with a lowly employee, which should not be possible for a `Pair<Manager>`.



Note: You just saw an important difference between generic types and Java arrays. You can assign a `Manager[]` array to a variable of type `Employee[]`:

```
Manager[] managerBuddies = { ceo, cfo };
Employee[] employeeBuddies = managerBuddies; // OK
```

However, arrays come with special protection. If you try to store a lowly employee into `employeeBuddies[0]`, the virtual machine throws an `ArrayStoreException`.

You can always convert a parameterized type to a raw type. For example, `Pair<Employee>` is a subtype of the raw type `Pair`. This conversion is necessary for interfacing with legacy code.

Can you convert to the raw type and then cause a type error? Indeed, you can. Consider this example:

```
var managerBuddies = new Pair<Manager>(ceo, cfo);
Pair rawBuddies = managerBuddies; // OK
rawBuddies.setFirst(new File(". . .")); // only a compile-time warning
```

This sounds scary. However, keep in mind that the security of the virtual machine is not at stake. When the foreign object is retrieved with `getFirst` and assigned to a `Manager` variable, a `ClassCastException` is thrown.

Finally, generic classes can extend or implement other generic classes. In this regard, they are no different from ordinary classes. For example, the class `ArrayList<T>` implements the interface `List<T>`. That means an `ArrayList<Manager>` can be converted to a `List<Manager>`. However, as you just saw, an `ArrayList<Manager>` is *not* an `ArrayList<Employee>` or `List<Employee>`. **Figure 8.2** shows these relationships.

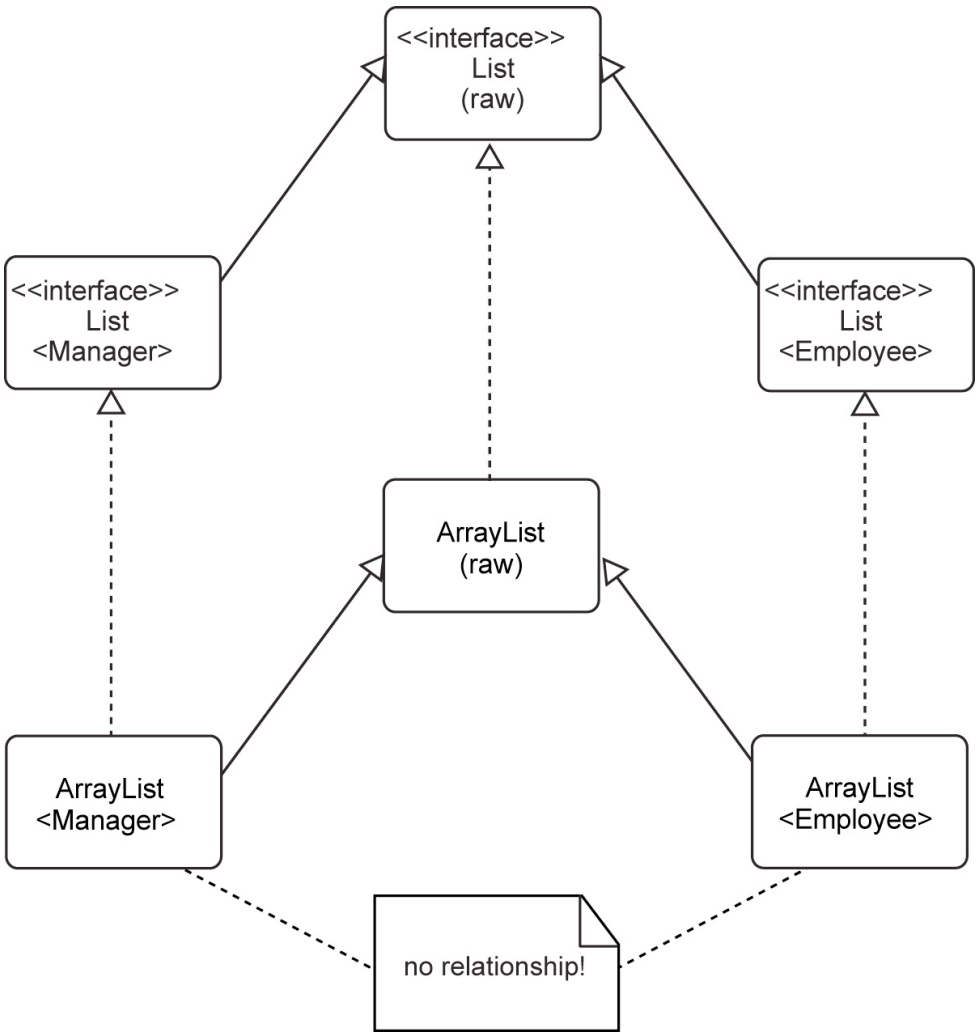


Figure 8.2: Subtype relationships among generic list types

8.4. Wildcard Types

It was known for some time among researchers of type systems that a rigid system of generic types is quite unpleasant to use. The Java designers invented an ingenious (but nevertheless safe) “escape hatch”: the *wildcard type*. The following sections show you how to work with wildcards.

8.4.1. The Wildcard Concept

In a wildcard type, a type parameter is allowed to vary. For example, the wildcard type

```
Pair<? extends Employee>
```

denotes any generic `Pair` type whose type parameter is a subclass of `Employee`, such as `Pair<Manager>`, but not `Pair<String>`.

Let's say you want to write a method that prints out pairs of employees, like this:

```
void printBuddies(Pair<Employee> p) {
    Employee first = p.getFirst();
    Employee second = p.getSecond();
    IO.println(first.getName() + " and " + second.getName() + " are buddies.");
}
```

As you saw in the preceding section, you cannot pass a `Pair<Manager>` to that method, which is rather limiting. But the solution is simple—use a wildcard type:

```
void printBuddies(Pair<? extends Employee> p)
```

The type `Pair<Manager>` is a subtype of `Pair<? extends Employee>` (see **Figure 8.3**).

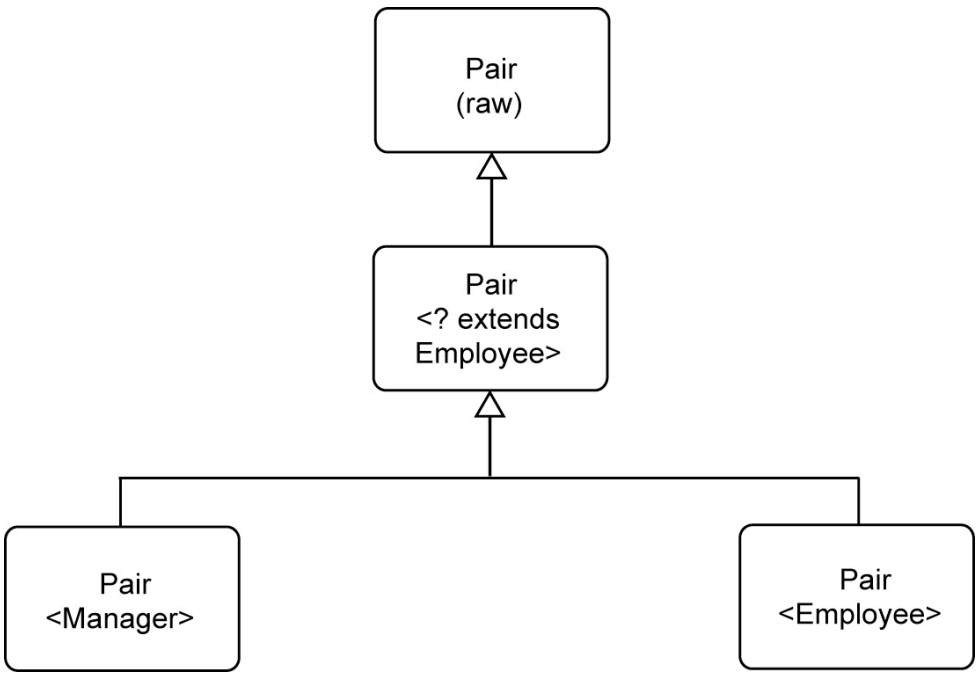


Figure 8.3: Subtype relationships with wildcards

Can we use wildcards to corrupt a `Pair<Manager>` through a `Pair<? extends Employee>` reference?

```
var managerBuddies = new Pair<Manager>(ceo, cfo);
Pair<? extends Employee> wildcardBuddies = managerBuddies; // OK
wildcardBuddies.setFirst(lowlyEmployee); // compile-time error
```

No corruption is possible. The call to `setFirst` is a type error. To see why, let us have a closer look at the type `Pair<? extends Employee>`. Its methods look like this:

```
? extends Employee getFirst()
void setFirst(? extends Employee)
```

It is impossible to call the `setFirst` method! Consider the call `wildcardBuddies.setFirst(lowlyEmployee)` The compiler knows that the parameter of `setFirst` has some specific type, which extends `Employee`. Is that specific type `Employee`? Is it `Manager`, or some other subclass? There is no way for the compiler to know. Therefore, the compiler cannot accept `lowlyEmployee`. For the same reason, the call `wildcardBuddies.setFirst(cio)`, where `cio` is a `Manager` instance, also fails. The compiler must reject all arguments to `setFirst` other than `null`.

The `getFirst` method continues to work. The return value of `getFirst` is an instance of some specific type, which is a subtype of `Employee`. The compiler doesn't know what that specific type is, but it can guarantee that the assignment to an `Employee` reference is safe.

This is the key idea behind bounded wildcards. We now have a way of distinguishing between the safe accessor methods and the unsafe mutator methods.

8.4.2. Supertype Bounds for Wildcards

Wildcard bounds are similar to type variable bounds, but they have an added capability—you can specify a *supertype bound*, like this:

```
? super Executive
```

This wildcard is restricted to all supertypes of `Executive`. (It was a stroke of good luck that the existing `super` keyword describes the relationship so accurately.)

Why would you want to do this? A wildcard with a supertype bound gives you a behavior that is opposite to that of the wildcards described in **Section 8.4.1**. You can supply arguments to methods, but you can't use the return values. For example, `Pair<? super Executive>` has methods that can be described as follows:

```
void setFirst(? super Executive)
? super Executive getFirst()
```

This is not the actual Java syntax, but it shows what the compiler knows. The `setFirst` parameter type, denoted as `? super Executive`, is some specific type T, and `Executive` is a subtype of T. There are exactly four choices for T: `Object`, `Employee`, `Manager`, or `Executive`. (There would have been more

choices if any of these classes had implemented interfaces.) However, the compiler cannot know which of these choices applies. Therefore, the compiler cannot accept a call with an argument of type `Employee` or `Object`. After all, `T` might have been `Manager`.

Conversely, if you call `getFirst`, there is no guarantee about the type of the returned object. You can only assign it to an `Object`.

This is all rather abstract, so let us look at a concrete situation. We have an array of executives and want to put the executive with the shortest and longest title into a `Pair` object. What kind of `Pair`? A `Pair<Executive>` is the obvious choice, but there are other possibilities. A `Pait<Manager>` or `Pair<Employee>` should be fair game or, for that matter, a `Pair<Object>` (see **Figure 8.4**). The following method will accept any appropriate `Pair`:

```
void minmaxTitle(Executive[] a, Pair<? super Executive> result) {
    if (a.length == 0) return;
    Executive smallest = a[0];
    Executive largest = a[0];
    for (int i = 1; i < a.length; i++) {
        if (smallest.getTitle().length() > a[i].getTitle().length()) smallest = a[i];
        if (largest.getTitle().length() < a[i].getTitle().length()) largest = a[i];
    }
    result.setFirst(smallest);
    result.setSecond(largest);
}
```

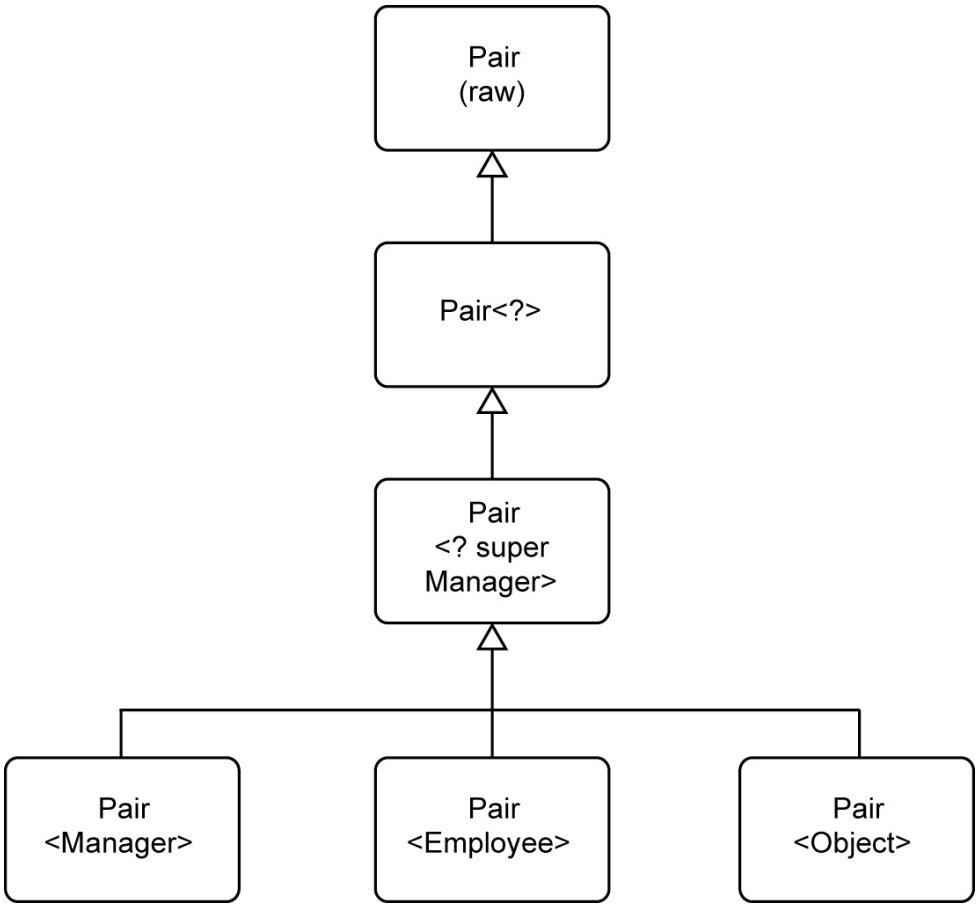


Figure 8.4: A wildcard with a supertype bound

Intuitively speaking, wildcards with supertype bounds let you write to a generic object, while wildcards with subtype bounds let you read from a generic object.

Here is another use for supertype bounds. The `Comparable` interface is itself a generic type. It is declared as follows:

```
public interface Comparable<T> {
    public int compareTo(T other);
}
```

Here, the type variable indicates the type of the `other` parameter. For example, the `String` class implements `Comparable<String>`, and its `compareTo` method is declared as

```
public int compareTo(String other)
```

This is nice—the explicit parameter has the correct type. Before the interface was generic, `other` was an `Object`, and a cast was necessary in the implementation of the method.

Now that `Comparable` is a generic type, perhaps we should have done a better job with the `minmax` method? We could have declared it as

```
<T extends Comparable<T>> Pair<T> minmax(T[] a)
```

This looks more thorough than just using `T extends Comparable`, and it would work fine for many classes. For example, if you compute the minimum of a `String` array, then `T` is the type `String`, and `String` is a subtype of `Comparable<String>`. But we run into a problem when processing an array of `LocalDate` objects. As it happens, `LocalDate` implements `ChronoLocalDate`, and `ChronoLocalDate` extends `Comparable<ChronoLocalDate>`. Thus, `LocalDate` implements `Comparable<ChronoLocalDate>` but *not* `Comparable<LocalDate>`.

In a situation such as this one, supertypes come to the rescue:

```
<T extends Comparable<? super T>> Pair<T> minmax(T[] a)
```

Now the `compareTo` method has the form

```
int compareTo(? super T)
```

Maybe it is declared to take an object of type `T`, or—for example, when `T` is `LocalDate`—a supertype of `T`. At any rate, it is safe to pass an object of type `T` to the `compareTo` method.

To the uninitiated, a declaration such as `<T extends Comparable<? super T>>` is bound to look intimidating. This is unfortunate, because the intent of this declaration is to help application programmers by removing unnecessary restrictions on the call arguments. Application programmers with no interest in generics will probably learn quickly to gloss over these declarations and just take for granted that library programmers will do the right thing. If you are a library programmer, you'll need to get used to wildcards, or your users will curse you and throw random casts at their code until it compiles.



Note: Another common use for supertype bounds is a parameter type of a functional interface. For example, the `Collection` interface has a method

```
default boolean removeIf(Predicate<? super E> filter)
```

The method removes all elements that fulfill the given predicate. For example, if you hate employees with odd hash codes, you can remove them like this:

```
ArrayList<Employee> staff = . . .;
Predicate<Object> oddHashCode = obj -> obj.hashCode() %2 != 0;
staff.removeIf(oddHashCode);
```

You want to be able to pass a `Predicate<Object>`, not just a `Predicate<Employee>`. The `super` wildcard makes that possible.

8.4.3. Unbounded Wildcards

You can even use wildcards with no bounds at all—for example, `Pair<?>`. At first glance, this looks identical to the raw `Pair` type. Actually, the types are very different. The type `Pair<?>` has methods such as

```
? getFirst()
void setFirst(?)
```

The return value of `getFirst` can only be assigned to an `Object`. The `setFirst` method can never be called, *not even with an* `Object`. That's the essential difference between `Pair<?>` and `Pair`: you can call the `setFirst` method of the raw `Pair` class with *any* `Object`.



Note: You can call `setFirst(null)`.

Why would you ever want such a wimpy type? It is useful for very simple operations. For example, the following method tests whether a pair contains a `null` reference. It never needs the actual type.

```
boolean hasNulls(Pair<?> p) {
    return p.getFirst() == null || p.getSecond() == null;
}
```

You could have avoided the wildcard type by turning `hasNulls` into a generic method:

```
<T> boolean hasNulls(Pair<T> p)
```

However, the version with the wildcard type seems easier to read.

8.4.4. Wildcard Capture

Let us write a method that swaps the elements of a pair:

```
void swap(Pair<?> p)
```

A wildcard is not a type variable, so we can't write code that uses `?` as a type. In other words, the following would be illegal:

```
? t = p.getFirst(); // ERROR
p.setFirst(p.getSecond());
p.setSecond(t);
```

That's a problem because we need to temporarily hold the first element when we do the swapping. Fortunately, there is an interesting solution to this problem. We can write a helper method, `swapHelper`, like this:

```
<T> void swapHelper(Pair<T> p) {
    T t = p.getFirst();
    p.setFirst(p.getSecond());
    p.setSecond(t);
}
```

Note that `swapHelper` is a generic method, whereas `swap` is not—it has a fixed parameter of type `Pair<?>`.

Now we can call `swapHelper` from `swap`:

```
void swap(Pair<?> p) { swapHelper(p); }
```

In this case, the parameter `T` of the `swapHelper` method *captures the wildcard*. It isn't known what type the wildcard denotes, but it is a definite type, and the definition of `<T>swapHelper` makes perfect sense when `T` denotes that type.

Of course, in this case, we were not compelled to use a wildcard. We could have directly implemented `<T> void swap(Pair<T> p)` as a generic method without wildcards. However, consider this example in which a wildcard type occurs naturally in the middle of a computation:

```
void maxminTitle(Executive[] a, Pair<? super Executive> result) {
    minmaxTitle(a, result);
    swap(result); // OK—swapHelper captures wildcard type
}
```

Here, the wildcard capture mechanism cannot be avoided.

Wildcard capture is only legal in very limited circumstances. The compiler must be able to guarantee that the wildcard represents a single definite type. For example, the `T` in `ArrayList<Pair<T>>` can never capture the wildcard in `ArrayList<Pair<?>>`. The array list might hold several `Pair<?>`, each of which has a different type for `?`.

The test program in **Listing 8.3** gathers up the various methods discussed in the preceding sections so you can see them in context.

Listing 8.3 v1ch08/PairDemo3.java

```
1  package v1ch08;
2
3  import com.horstmann.corejava.*;
4
5  class PairDemo3 {
6      void main() {
7          var ceo = new Executive("Gus Greedy", "CEO",
8                  800000, 2003, 12, 15);
9          var savp = new Executive("Sue Striver", "Senior Associate Vice President",
10                 200000, 1995,1, 20);;
11         var buddies = new Pair<Manager>(ceo, savp);
12         printBuddies(buddies);
13
14         Executive[] executives = { ceo, savp };
15
16         var result = new Pair<Employee>();
17         IO.println("hasNulls: " + hasNulls(result));
18         minmaxTitle(executives, result);
19         IO.println("first: " + result.getFirst().getName() + ", second: "
20                 + result.getSecond().getName());
21         maxminTitle(executives, result);
22         IO.println("first: " + result.getFirst().getName() + ", second: "
23                 + result.getSecond().getName());
24     }
25
26     void printBuddies(Pair<? extends Employee> p) {
27         Employee first = p.getFirst();
28         Employee second = p.getSecond();
29         IO.println(first.getName() + " and " + second.getName() + " are buddies.");
```

```
    }

    void minmaxTitle(Executive[] a, Pair<? super Executive> result) {
        if (a.length == 0) return;
        Executive smallest = a[0];
        Executive largest = a[0];
        for (int i = 1; i < a.length; i++) {
            if (smallest.getTitle().length() > a[i].getTitle().length()) smallest =
            if (largest.getTitle().length() < a[i].getTitle().length()) largest = a[
        }
        result.setFirst(smallest);
        result.setSecond(largest);
    }

    void maxminTitle(Executive[] a, Pair<? super Executive> result) {
        minmaxTitle(a, result);
        swap(result); // OK--swapHelper captures wildcard type
    }

    boolean hasNulls(Pair<?> p) {
        return p.getFirst() == null || p.getSecond() == null;
    }

    void swap(Pair<?> p) {
        swapHelper(p);
    }

    <T> void swapHelper(Pair<T> p) {
        T t = p.getFirst();
        p.setFirst(p.getSecond());
        p.setSecond(t);
    }
}
```

8.5. Restrictions and Limitations

In the following sections, I discuss a number of restrictions that you need to consider when working with Java generics. Most of these restrictions are a consequence of type erasure.

8.5.1. Type Parameters Cannot Be Instantiated with Primitive Types

You cannot substitute a primitive type for a type parameter. Thus, there is no `Pair<double>`, only `Pair<Double>`. The reason is, of course, type erasure. After erasure, the `Pair` class has fields of type `Object`, and you can't use them to store `double` values.

This is an annoyance, to be sure, but it is consistent with the separate status of primitive types in the Java language. It is not a fatal flaw—there are only eight primitive types, and you can always handle them with separate classes and methods when wrapper types are not an acceptable substitute.

8.5.2. Casts Only Work with Raw Types

Objects in the virtual machine always have a specific nongeneric type. Therefore, all type inquiries yield only the raw type. For example, consider this cast:

```
Pair<String> p = (Pair<String>) obj; // warning--can only test that obj is some Pair
```

The cast will succeed if `obj` is a `Pair` of any type.

Similarly, in a generic method, a cast

```
Pair<T> p = (Pair<T>) obj;
```

cannot actually test whether `obj` is a pair of the appropriate type. It will just check for a raw `Pair`.

Finally, a cast

```
T t = (T) obj;
```

does nothing at runtime.



Note: The casts compile with warnings, but the corresponding `instanceof` tests are compile-time errors:

```
if (obj instanceof Pair<String> p) // ERROR
if (obj instanceof Pair<T> p) // ERROR
if (obj instanceof T) // ERROR
```

Why do you need such casts at all? Consider the following example:

```
<T> T defaultValue(Class<T> cl) {
    if (cl == String.class) return ""; // ERROR
    . . .;
}
```

The compiler complains that the result type is `T`, but `""` is a `String`. You have to use a cast to keep the compiler happy:

```
<T> T defaultValue(Class<T> cl) {
    if (cl == String.class) return (T) "";
    . . .;
}
```



Note: It is surprisingly difficult to handle primitive types in this situation. You need two or even three casts:

```
if (cl == Integer.class) return (T) (Integer) 0;
if (cl == Character.class) return (T) (Character) (char) 0;
```

8.5.3. You Cannot Create Arrays of Parameterized Types

You cannot instantiate arrays of parameterized types, such as

```
var table = new Pair<String>[10]; // ERROR
```

What's wrong with that? After erasure, the type of `table` is `Pair[]`. You can convert it to `Object[]`:

```
Object[] objarray = table;
```

An array remembers its component type and throws an `ArrayStoreException` if you try to store an element of the wrong type:

```
objarray[0] = "Hello"; // ERROR--component type is Pair
```

But erasure renders this mechanism ineffective for generic types. The assignment

```
objarray[0] = new Pair<Employee>();
```

would pass the array store check but still result in a type error. For this reason, arrays of parameterized types are outlawed.

Note that only the creation of these arrays is outlawed. You can declare a variable of type `Pair<String>[]`. But you can't initialize it with a `new Pair<String>[10]`.



Note: You can declare arrays of wildcard types and then cast them:

```
var table = (Pair<String>[]) new Pair<?>[10];
```

The result is not safe. If you store a `Pair<Employee>` in `table[0]` and then call a `String` method on `table[0].getFirst()`, you get a `ClassCastException`.



Tip: If you need to collect parameterized type objects, simply use an `ArrayList`: `ArrayList<Pair<String>>` is safe and effective.

8.5.4. Varargs Warnings

In the preceding section, you saw that Java doesn't support arrays of generic types. In this section, I discuss a related issue: passing instances of a generic type to a method with a variable number of arguments.

Consider this simple method with variable arguments:

```
<E> void addAll(Collection<E> coll, E... elements) {
    for (E element : elements) coll.add(element);
}
```

Recall that the parameter `elements` is actually an array that holds all supplied arguments.

Now consider this call:

```
Collection<Pair<String>> table = . . .;
Pair<String> pair1 = . . .;
Pair<String> pair2 = . . .;
addAll(table, pair1, pair2);
```

In order to call this method, the Java virtual machine must make an array of `Pair<String>`, which is against the rules. However, the rules have been relaxed for this situation, and you only get a warning at the call site, not an error. You can suppress that warning by adding the annotation `@SuppressWarnings("unchecked")` to the method containing the call to `addAll`. However, this is an annoying burden for the callers of the `addAll` method. For that reason, there is another warning about "possible heap pollution" at the declaration site.

To avoid both warnings, annotate the `addAll` method itself with `@SafeVarargs`:

```
@SafeVarargs
static <E> void addAll(Collection<E> coll, E... elements)
```

This method can now be called with generic types. By using the annotation, you promise that your method doesn't write to the parameter array, and doesn't pass the array to others who might do that. See the note below for an example of abuse.

The `@SafeVarargs` annotation can only be used with constructors and methods that are `static`, `final`, or `private`. Any other method could be overridden, making the annotation meaningless.



Caution: You can use the `@SafeVarargs` annotation to defeat the restriction against generic array creation, using this method:

```
@SafeVarargs static <E> E[] array(E... array) { return array; }
```

Now you can call

```
Pair<String>[] table = array(pair1, pair2);
```

This seems convenient, but there is a hidden danger. The code

```
Object[] objarray = table;
objarray[0] = new Pair<Employee>();
```

will run without an `ArrayStoreException` because the array store only checks the erased type. You'll get an exception elsewhere when you work with `table[0]`.

8.5.5. Generic Varargs Do Not Spread Primitive Arrays

Recall that you can pass an array to a varargs parameter. For example, consider the `addAll` method of the preceding section. When the items to be added happen to be in an array, you just pass the array. The call behaves as if the elements had been "spread out":

```
Collection<String> strings = . . .;
String[] moreStrings = new String[] { "Mary", "had", "a", "little", "lamb" };
addAll(strings, moreStrings);
    // OK, same as addAll(strings, "Mary", "had", "a", "little", "lamb")
```

Of course, the elements are not actually spread out. In fact, the converse is true. If the arguments had been individually specified, they would be gathered in an array, which would then be passed to the method. If you supply an array, it is passed directly.

Now consider a collection of numbers:

```
Collection<Integer> numbers = . . .;
int[] moreNumbers = new int[] { 11, 12, 13, 14, 15 };
addAll(numbers, moreNumbers); // ERROR
```

This does not work. There is no valid match for the type variable `E`. An `int[]` cannot be converted to an `Integer[]`.

Here is a more insidious example. The `List` interface has a static `of` method:

```
public static <E> List<E> of(E... elements)
```

The call

```
List.of(moreStrings) // ["Mary", "had", "a", "little", "lamb"]
```

yields a list of 5 elements. But

```
List.of(moreNumbers)
```

yields a `List<int[]>` of a single element, the array of five `int` values. The type parameter `E` cannot match the primitive type `int`, so the `moreNumbers` elements are not spread. Instead, the type parameter `E` matches the array type `int[]`.

8.5.6. You Cannot Instantiate Type Variables

You cannot use type variables in an expression such as `new T(. . .)`. For example, the following method is illegal:

```
<T> Pair<T> makePair() { return new Pair<T>(new T(); new T()); } // ERROR
```

Type erasure would change `T` to `Object`, and surely you don't want to call `new Object()`.

A common workaround in modern Java is to make the caller provide a constructor expression. For example:

```
Pair<String> p = makePair(String::new);
```

The `makePair` method receives a `Supplier<T>`, the functional interface for a function with no parameters and a result of type `T`:

```
<T> Pair<T> makePair(Supplier<T> constr) {
    return new Pair<>(constr.get(), constr.get());
}
```

A more traditional workaround is to construct generic objects through reflection, by calling the `Constructor.newInstance` method.

Unfortunately, the details are a bit complex. You cannot call

```
first = T.class.getConstructor().newInstance(); // ERROR
```

The expression `T.class` is not legal because it would erase to `Object.class`. Instead, you must design the API so that you are handed a `Class` object, like this:

```
<T> Pair<T> makePair(Class<T> cl) throws ReflectiveOperationException {
    Constructor<T> constr = cl.getConstructor();
    return new Pair<>(constr.newInstance(), constr.newInstance());
}
```

The method is declared to throw a `ReflectiveOperationException` because the class might not have a no-argument constructor.

When calling the method, provide a class literal as follows:

```
Pair<String> p = Pair.makePair(String.class);
```

Note that the `Class` class is itself generic. For example, `String.class` is an instance (indeed, the sole instance) of `Class<String>`. Therefore, the `makePair` method can infer the type of the pair that it is making.

Another workaround makes use of variable arguments. This method has a varargs parameter that is not intended to be used. It's only purpose is to provide the component type.

```
<T> Pair<T> makePair(T... dummy) throws ReflectiveOperationException {
    Constructor<?> constr = dummy.getClass().getComponentType().getConstructor();
    return new Pair<>((T) constr.newInstance(), (T) constr.newInstance());
}
```

When you call

```
Pair<String> names = makePair();
```

the method `<String>makePair` is called with an empty array of type `String[]`. That array yields the component type `String`.

A drawback is the confusing API. One would need to explain to users that the method is intended to be invoked with no arguments.

8.5.7. You Cannot Construct a Generic Array

Just as you cannot instantiate a single generic instance, you cannot instantiate an array. The reasons are different—an array is, after all, filled with `null` values, which would seem safe to construct. But an array also carries a type, which is used to monitor array stores in the virtual machine. That type is erased. For example, consider

```
<T extends Comparable> T[] minmax(T... a) {
    T[] mm = new T[2]; // ERROR
    . . .
}
```

Type erasure would cause this method to always construct an array `Comparable[2]`.

If the array is only used as a private instance field of a class, you can declare the element type of the array to be the erased type and use casts. For example, the `ArrayList` class can be implemented as follows:

```
public class ArrayList<E> {
    private Object[] elements;
    . . .
    @SuppressWarnings("unchecked") public E get(int n) { return (E) elements[n]; }
    public void set(int n, E e) { elements[n] = e; } // no cast needed
}
```

This technique does not work for our `minmax` method since we are returning a `T[]` array, and a runtime error results if we lie about its type. Suppose we implement

```
<T extends Comparable> T[] minmax(T... a) {
    var result = new Comparable[2]; // array of erased type
    . . .
    return (T[]) result; // compiles with warning
}
```

The call

```
String[] names = minmax("Tom", "Dick", "Harry");
```

compiles without any warning. A `ClassCastException` occurs when the `Comparable[]` reference is cast to `String[]` after the method returns.

In this situation, it is best to ask the user to provide an array constructor expression:

```
String[] names = minmax(String[]::new, "Tom", "Dick", "Harry");
```

The constructor expression `String[]::new` denotes a function that, given the desired length, constructs a `String` array of that length.

The method uses that parameter to produce an array of the correct type:

```
<T extends Comparable> T[] minmax(IntFunction<T[]> constr, T... a) {
    T[] result = constr.apply(2);
    . . .
}
```

A more old-fashioned approach is to use reflection and call `Array.newInstance`:

```
<T extends Comparable> T[] minmax(T... a) {
    var result = (T[]) Array.newInstance(a.getClass().getComponentType(), 2);
    . . .
}
```

The `toArray` method of the `ArrayList` class is not so lucky. It needs to produce a `T[]` array, but it doesn't have the component type. Therefore, there are two variants:

```
Object[] toArray()
T[] toArray(T[] result)
```

The second method receives an array argument. If the array is large enough, it is used. Otherwise, a new array of sufficient size is created, using the component type of `result`.

8.5.8. Type Variables Are Not Valid in Static Contexts of Generic Classes

You cannot reference type variables in static fields or methods. For example, the following clever idea won't work:

```
public class Singleton<T> {
    private static T singleInstance; // ERROR

    public static T getSingleInstance() { // ERROR
        if (singleInstance == null) construct new instance of T
        return singleInstance;
    }
}
```

If this could be done, then a program could declare a `Singleton<Random>` to share a random number generator and a `Singleton<JFileChooser>` to share a file chooser dialog. But it can't work. After type erasure there is only one `Singleton` class, and only one `singleInstance` field. For that reason, static fields and methods with type variables are simply outlawed.

8.5.9. You Can Defeat Checked Exception Checking

A bedrock principle of Java exception handling is that you must provide a handler for all checked exceptions. You can use generics to defeat this scheme. The key ingredient is this method:

```
class Sneaky {
    @SuppressWarnings("unchecked")
    static <T extends Throwable> void throwAs(Throwable t) throws T {
        throw (T) t;
    }
    . . .
}
```

When you have a checked exception `e` and call

```
Sneaky.<RuntimeException>throwAs(e);
```

then the compiler will believe that `e` becomes an unchecked exception.

In fact, the exception stays checked. The cast `(T) t` is meaningless and does not lead to any check at runtime.

Note that you must use generics. You cannot simply call

```
throw (RuntimeException) t; // Does not work
```

Then there would be a check at runtime, and a `ClassCastException` would occur.

Let's use this to solve a vexing problem. To run code in a thread, you have to place it into the `run` method of a class that implements the `Runnable` interface. But that method is not allowed to throw checked exceptions. We will provide an adaptor from a `Sneaky.Runnable`, whose `run` method is allowed to throw arbitrary exceptions, to a `Runnable`:

```
class Sneaky {
    . . .
    interface Runnable { void run() throws Exception; }

    static java.lang.Runnable asRunnable(Sneaky.Runnable task) {
        return () -> {
            try {
                task.run();
            }
            catch (Exception e) {
                Sneaky.<RuntimeException>throwAs(e);
            }
        };
    }
}
```

For example, this program runs a thread that will throw a checked exception:

```
void main() {
    var thread = new Thread(Sneaky.asRunnable(() -> {
        Thread.sleep(1000);
        IO.println("Hello, World!");
        throw new Exception("Check this out!");
    }));
    thread.start();
}
```

The `Thread.sleep` method is declared to throw an `InterruptedException`, and we no longer have to catch it. Since we don't interrupt the thread, that exception won't be thrown. However, the program throws a checked exception. When you run the program, you will get a stack trace.

What's so remarkable about that? Normally, you have to catch all checked exceptions inside the `run` method of a `Runnable` and *wrap them* into unchecked exceptions—the `run` method is declared to throw no checked exceptions.

But here, we don't wrap into an unchecked exception. We simply rethrow the checked exception, tricking the compiler into believing that it is a `RuntimeException`.

Using generic classes and erasure, we were able to defeat an essential part of the Java type system.

8.5.10. Beware of Clashes after Erasure

It is illegal to create conditions that cause clashes when generic types are erased. Here is an example. Suppose we add an `equals` method to the `Pair` class, like this:

```
public class Pair<T> {
    public boolean equals(T value) { return first.equals(value) && second.equals(value); }
    . . .
}
```

Consider a `Pair<String>`. Conceptually, it has two `equals` methods:

```
boolean equals(String) // defined in Pair<T>
boolean equals(Object) // inherited from Object
```

But the intuition leads us astray. The erasure of the method

```
boolean equals(T)
```

is

```
boolean equals(Object)
```

which clashes with the `Object.equals` method.

The remedy is, of course, to rename the offending method.

The generics specification cites another rule: “To support translation by erasure, we impose the restriction that a class or type variable may not at the same time be a subtype of two interface types which are different parameterizations of the same interface.” For example, the following is illegal:

```
class Employee implements Comparable<Employee> { . . . }
class Manager extends Employee implements Comparable<Manager> { . . . } // ERROR
```

`Manager` would then implement both `Comparable<Employee>` and `Comparable<Manager>`, which are different parameterizations of the same interface.

It is not obvious what this restriction has to do with type erasure. After all, the nongeneric version

```
class Employee implements Comparable { . . . }
class Manager extends Employee implements Comparable { . . . }
```

is legal. The reason is far more subtle. There would be a conflict with the synthesized bridge methods. A class that implements `Comparable<X>` gets a bridge method

```
public int compareTo(Object other) { return compareTo((X) other); }
```

You cannot have two such methods for different types `X`.

8.5.11. Type Inference in Generic Record Patterns is Limited

Let us make our `Pair` class into a record:

```
record Pair<T>(T first, T second) {}
```

Now you can form a record pattern with the generic record:

```
var p = new Pair<String>("Hello", "World");
String content = switch (p) {
    case Pair(var a, var b) -> a + " " + b.toUpperCase();
};
```



Note: In the pattern, the type argument is inferred—here, as `Pair<String>`. You can also be more explicit:

```
 case Pair<String>(var a, var b)
 case Pair(String a, String b)
 case Pair<String>(String a, String b)
```

When generic types are involved, the compiler may need to work pretty hard to verify exhaustiveness of a `switch`. Consider this incomplete hierarchy of JSON types:

```
sealed interface JSONPrimitive<T> {}
record JSONNumber(double value) implements JSONPrimitive<Double> {}
record JSONBoolean(boolean value) implements JSONPrimitive<Boolean> {}
record JSONString(String value) implements JSONPrimitive<String> {}
```

The `switch` in the following method is exhaustive:

```
<T> double toNumber(JSONPrimitive<T> v) {
    return switch (v) {
        case JSONNumber(var n) -> n;
        case JSONBoolean(var b) -> b ? 1 : 0;
        case JSONString(var s) -> {
            try {
                yield Double.parseDouble(s);
            }
            catch (NumberFormatException ex) {
                yield Double.NaN;
            }
         }
     };
}
```

At first glance, it appears as if there might be an unbounded number of classes implementing `JSONPrimitive<T>`, but the compiler can track that there are only three of them.

Now we want to form the sum of two JSON primitives:

```
Object sum(Pair<? extends JSONPrimitive<?>> pair) {
    return switch (pair) {
        case Pair(JSONNumber(var left), JSONNumber(var right)) -> left + right;
        case Pair(JSONBoolean(var left), JSONBoolean(var right)) -> left | right;
        case Pair(JSONString(var left), JSONString(var right)) -> left.concat(right);
        // ERROR--not exhaustive
    };
}
```

This `switch` is not exhaustive. After all, it would be possible to call `sum1` as

```
sum(Pair.of(new JSONNumber(42), new JSONString("Fred")))
```

You can make the `switch` exhaustive by adding the other six combinations of JSON primitives and throwing an exception in each case. But that is a runtime check. It would be nicer to reject mixed pairs at compile time.

Here is how to only accept homogeneous pairs:

```
<T extends JSONPrimitive<U>, U> Object sum(Pair<T> pair) {
    return switch (pair) {
        case Pair(JSONNumber(var left), JSONNumber(var right)) -> left + right;
        case Pair(JSONBoolean(var left), JSONBoolean(var right)) -> left | right;
        case Pair(JSONString(var left), JSONString(var right)) -> left.concat(right);
        default -> throw new AssertionError(); // Sadly Java can't tell this won't happen
    };
}
```

Now the call

```
sum(Pair.of(new JSONNumber(42), new JSONString("Fred")))
```

no longer compiles, since there are no matching types for `T` and `U`.

Unfortunately, the `default` clause is necessary to make the `switch` exhaustive. In theory, there is enough information to determine that the pair components must be instances of the same type, but the Java type system can't prove it.



Caution: Trying to use explicit type arguments does not work:

```
<T extends JSONPrimitive<U>, U> Object sum(Pair<T> pair) {
    return switch (pair) {
        // ERROR—unsafe casts
        case Pair<JSONNumber>(JSONNumber(var left), JSONNumber(var right))
            -> left + right;
        case Pair<JSONBoolean>(JSONBoolean(var left), JSONBoolean(var right))
            -> left | right;
        case Pair<JSONString>(JSONString(var left), JSONString(var right))
            -> left.concat(right);
    };
}
```

The Java compiler does not know how to prove that the cast from `Pair<T>` to `Pair<JSONNumber>` is safe when the components have type `JSONNumber`.

8.6. Reflection and Generics

Reflection lets you analyze arbitrary objects at runtime. If the objects are instances of generic classes, you don't get much information about the generic type parameters because they have been erased. In the following sections, you will learn what you can nevertheless find out about generic classes with reflection.

8.6.1. The Generic `Class` Class

The `Class` class has a type parameter. For example, `String.class` is actually an object (in fact, the sole object) of the class `Class<String>`.

The type parameter is useful because it allows the methods of `Class<T>` to be more specific about their return types. The following methods of `Class<T>` take advantage of the type parameter:

```
T cast(Object obj)
T[] getEnumConstants()
Class<? super T> getSuperclass()
Constructor<T> getConstructor(Class<?>... parameterTypes)
Constructor<T> getDeclaredConstructor(Class<?>... parameterTypes)
```

The `getConstructor` method returns an instance of `Constructor<T>`, so that no cast is required in

```
Class<T> cl = . . .;
T noArgInstance = cl.getConstructor().newInstance();
```

The `cast` method returns the given object, now declared as type `T` if its type is indeed a subtype of `T`. Otherwise, it throws a `ClassCastException`.

The `getEnumConstants` method returns `null` if this class is not an `enum` class or an array of the enumeration values which are known to be of type `T`.

`java.lang.Class<T>` 1.0

- `T cast(Object obj)`
  returns `obj` if it is `null` or can be converted to the type `T`, or throws a `ClassCastException` otherwise.
- `T[] getEnumConstants()` 5.0
  returns an array of all values if `T` is an enumerated type, `null` otherwise.
- `Class<? super T> getSuperclass()`
  returns the superclass of this class, or `null` if `T` is not a class or the class `Object`.
- `Constructor<T> getConstructor(Class... parameterTypes)` 1.1
- `Constructor<T> getDeclaredConstructor(Class...`

`parameterTypes)` 1.1
get the public constructor, or the constructor with the given parameter types.

8.6.2. Using `Class<T>` Parameters for Type Matching

It is sometimes useful to match the type variable of a `Class<T>` parameter in a generic method. Here is the canonical example:

```
public static <T> Pair<T> makePair(Class<T> c) throws ReflectiveOperationException {
    Constructor<T> constr = c.getConstructor();
    return new Pair<>(constr.newInstance(), constr.newInstance());
}
```

If you call

```
makePair(String.class)
```

then `String.class` is an object of type `Class<String>`. The type parameter `T` of the `makePair` method matches `String`, and the compiler can infer that the method returns a `Pair<String>`.

8.6.3. Generic Type Information in the Virtual Machine

One of the notable features of Java generics is the erasure of generic types in the virtual machine. Perhaps surprisingly, the erased classes still retain some faint memory of their generic origin. For example, the raw `Pair` class knows that it originated from the generic class `Pair<T>`, even though an object of type `Pair` can't tell whether it was constructed as a `Pair<String>` or `Pair<Employee>`.

Similarly, consider a method

```
Comparable min(Comparable[] a)
```

that is the erasure of a generic method

```
<T extends Comparable<? super T>> T min(T[] a)
```

You can use the reflection API to determine that

- The generic method has a type parameter called `T`;
- The type parameter has a subtype bound that is itself a generic type;
- The bounding type has a wildcard parameter;
- The wildcard parameter has a supertype bound; and
- The generic method has a generic array parameter.

In other words, you can reconstruct everything about generic classes and methods that their implementors declared. However, you won't know how the type parameters were resolved for specific objects or method calls.

In order to express generic type declarations, use the interface `Type` in the `java.lang.reflect` package. The interface has the following subtypes:

- The `Class` class, describing concrete types
- The `TypeVariable` interface, describing type variables (such as `T extends Comparable<? super T>`)
- The `WildcardType` interface, describing wildcards (such as `? super T`)
- The `ParameterizedType` interface, describing generic class or interface types (such as `Comparable<? super T>`)
- The `GenericArrayType` interface, describing generic arrays (such as `T[]`)

Figure 8.5 shows the inheritance hierarchy. Note that the last four subtypes are interfaces—the virtual machine instantiates suitable classes that implement these interfaces.

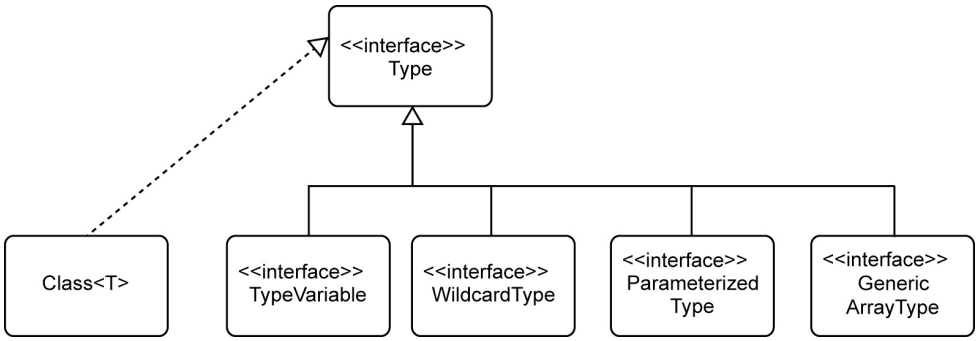


Figure 8.5: The `Type` interface and its descendants

The program in **Listing 8.4** and **Listing 8.5** uses the generic reflection API to print out what it discovers about a given class. If you run it with the `Pair` class, you get this report:

```
class com.horstmann.corejava.Pair<T> extends java.lang.Object
public T getFirst()
public T getSecond()
public void setFirst(T)
public void setSecond(T)
```

If you run it with `java.util.Collections`, the report displays methods with impressive generic types, such as:

```
public static <T extends java.lang.Object & java.lang.Comparable<? super T>>
T min(java.util.Collection<? extends T>)
```

Listing 8.4 v1ch08/genericReflection/GenericReflectionDemo.java

```
package v1ch08.genericReflection;

import module java.base;

class GenericReflectionDemo {
    void main(String[] args) throws Exception {
        // read class name from command line args or user input
        String name = args.length > 0 ? args[0]
                : IO.readln("Enter class name (e.g., java.util.Collections): ");

        // print generic info for class and public methods
        Class<?> cl = Class.forName(name);
        Printer.printClass(cl);
        for (Method m : cl.getDeclaredMethods())
            Printer.printMethod(m);
    }
}
```

Listing 8.5 v1ch08/genericReflection/Printer.java

```
package v1ch08.genericReflection;

import module java.base;

public class Printer {
    public static void printClass(Class<?> cl) {
        IO.print(cl);
        printTypes(cl.getTypeParameters(), "<", ", ", ">", true);
        Type sc = cl.getGenericSuperclass();
        if (sc != null) {
            IO.print(" extends ");
            printType(sc, false);
        }
        printTypes(cl.getGenericInterfaces(), " implements ", ", ", "", false);
        IO.println();
    }

    public static void printMethod(Method m) {
        String name = m.getName();
        IO.print(Modifier.toString(m.getModifiers()));
        IO.print(" ");
        printTypes(m.getTypeParameters(), "<", ", ", "> ", true);

        printType(m.getGenericReturnType(), false);
        IO.print(" ");
        IO.print(name);
        IO.print("(");
        printTypes(m.getGenericParameterTypes(), "", ", ", "", false);
        IO.println(")");
    }

    public static void printTypes(Type[] types, String prefix, String separator, Str
```

```
            boolean isDefinition) {
        if (prefix.equals(" extends ") && Arrays.equals(types, new Type[] { Object.c
            return;
        if (types.length > 0) IO.print(prefix);
        for (int i = 0; i < types.length; i++) {
            if (i > 0) IO.print(separator);
            printType(types[i], isDefinition);
        }
        if (types.length > 0) IO.print(suffix);
    }

    public static void printType(Type type, boolean isDefinition) {
        if (type instanceof Class<?> t) {
            IO.print(t.getName());
        }
        else if (type instanceof TypeVariable<?> t) {
            IO.print(t.getName());
            if (isDefinition) printTypes(t.getBounds(), " extends ", " & ", "", fals
        }
        else if (type instanceof WildcardType t) {
            IO.print("?");
            printTypes(t.getUpperBounds(), " extends ", " & ", "", false);
            printTypes(t.getLowerBounds(), " super ", " & ", "", false);
        }
        else if (type instanceof ParameterizedType t) {
            Type owner = t.getOwnerType();
            if (owner != null) {
                printType(owner, false);
                IO.print(".");
            }
            printType(t.getRawType(), false);
            printTypes(t.getActualTypeArguments(), "<", ", ", ">", false);
        }
        else if (type instanceof GenericArrayType t) {
            IO.print("");
            printType(t.getGenericComponentType(), isDefinition);
            IO.print("[]");
        }
    }
}
```

8.6.4. Type Literals

Sometimes, you want to drive program behavior by the type of a value. For example, in a persistence mechanism, you may want the user to specify a way of saving an object of a particular class. This is typically implemented by associating the `Class` object with an action.

However, with generic classes, erasure poses a problem. How can you have different actions for, say, `ArrayList<Integer>` and `ArrayList<String>` when both erase to the same raw `ArrayList` type?

There is a trick that can offer relief in some situations. You can capture an instance of the `Type` interface that you encountered in the preceding section.

Construct an anonymous subclass like this:

```
var type = new TypeLiteral<ArrayList<Integer>>(){} // note the {}
```

The `TypeLiteral` constructor captures the generic supertype:

```
class TypeLiteral<T> {
    public TypeLiteral() {
        Type parentType = getClass().getGenericSuperclass();
        if (parentType instanceof ParameterizedType paramType)
            type = paramType.getActualTypeArguments()[0];
        else
            throw new UnsupportedOperationException(
                "Construct as new TypeLiteral<. . .>(){}");
    }
    . . .
}
```

If we have a generic type available at runtime, we can match it against the `TypeLiteral`. We can't get a generic type from an object—it is erased. But, as you have seen in the preceding section, generic types of fields and method parameters survive in the virtual machine.

Injection frameworks such as CDI and Guice use type literals to control injection of generic types. The example program in the book's companion code shows a simpler example. Given an object, we enumerate its fields, whose generic types are available, and look up associated formatting actions.

We format an `ArrayList<Integer>` by separating the values with spaces, an `ArrayList<Character>` by joining the characters to a string. Any other array lists are formatted by `ArrayList.toString`.

Listing 8.6 v1ch08/genericReflection/TypeLiterals.java

```
1  package v1ch08.genericReflection;
2
3  import module java.base;
4
5  /**
6   * A type literal describes a type that can be generic, such as
7   * ArrayList<String>.
8   */
9  class TypeLiteral<T> {
10    private Type type;
11
12    /**
13     * This constructor must be invoked from an anonymous subclass as new
14     * TypeLiteral<. . .>(){}.
15     */
16    public TypeLiteral() {
17        Type parentType = getClass().getGenericSuperclass();
```

```
        if (parentType instanceof ParameterizedType paramType)
            type = paramType.getActualTypeArguments()[0];
        else
            throw new UnsupportedOperationException("Construct as new TypeLiteral<.
    }

    private TypeLiteral(Type type) {
        this.type = type;
    }

    /**
     * Yields a type literal that describes the given type.
     */
    public static TypeLiteral<?> of(Type type) {
        return new TypeLiteral<>(type);
    }

    public String toString() {
        if (type instanceof Class<?> cl)
            return cl.getName();
        else
            return type.toString();
    }

    public boolean equals(Object otherObject) {
        return otherObject instanceof TypeLiteral<?> otherLiteral
                && type.equals(otherLiteral.type);
    }

    public int hashCode() {
        return type.hashCode();
    }
}

/**
 * Formats objects, using rules that associate types with formatting functions.
 */
class Formatter {
    private Map<TypeLiteral<?>, Function<?, String>> rules = new HashMap<>();

    /**
     * Add a formatting rule to this formatter.
     * @param type the type to which this rule applies
     * @param formatterForType the function that formats objects of this type
     */
    public <T> void forType(TypeLiteral<T> type, Function<T, String> formatterForTyp
        rules.put(type, formatterForType);
    }

    /**
     * Formats all fields of an object using the rules of this formatter.
     * @param obj an object
     * @return a string with all field names and formatted values
     */
    public String formatFields(Object obj)
            throws IllegalArgumentException, IllegalAccessException {
```

```
        var result = new StringBuilder();
        for (Field f : obj.getClass().getDeclaredFields()) {
            result.append(f.getName());
            result.append("=");
            f.setAccessible(true);
            Function<?, String> formatterForType = rules
                    .get(TypeLiteral.of(f.getGenericType()));
            if (formatterForType != null) {
                // formatterForType has parameter type ?. Nothing can be passed to i
                // apply method. The cast makes the parameter type to Object so we c
                // invoke it.
                @SuppressWarnings("unchecked") Function<Object, String> objectFormat
                    = (Function<Object, String>) formatterForType;
                result.append(objectFormatter.apply(f.get(obj)));
            }
            else
                result.append(f.get(obj).toString());
            result.append("\n");
        }
        return result.toString();
    }
}

public class TypeLiterals {
    public static class Sample {
        ArrayList<Integer> nums;
        ArrayList<Character> chars;
        ArrayList<String> strings;

        public Sample() {
            nums = new ArrayList<>();
            nums.add(42);
            nums.add(1729);
            chars = new ArrayList<>();
            chars.add('H');
            chars.add('i');
            strings = new ArrayList<>();
            strings.add("Hello");
            strings.add("World");
        }
    }

    private static <T> String join(String separator, ArrayList<T> elements) {
        var result = new StringBuilder();
        for (T e : elements) {
            if (result.length() > 0) result.append(separator);
            result.append(e.toString());
        }
        return result.toString();
    }

    void main() throws Exception {
        var formatter = new Formatter();
        formatter.forType(new TypeLiteral<ArrayList<Integer>>() { },
                lst -> join(" ", lst));
        formatter.forType(new TypeLiteral<ArrayList<Character>>() { },
```

```
130                 lst -> "\"" + join("", lst) + "\"");
131         IO.println(formatter.formatFields(new Sample()));
132     }
133 }
```

`java.lang.Class<T>` 1.0

- `TypeVariable[] getTypeParameters()` 5.0
  gets the generic type variables if this type was declared as a generic type, or an array of length 0 otherwise.
- `Type getGenericSuperclass()` 5.0
  gets the generic type of the superclass that was declared for this type, or `null` if this type is `Object` or not a class type.
- `Type[] getGenericInterfaces()` 5.0
  gets the generic types of the interfaces that were declared for this type, in declaration order, or an array of length 0 if this type doesn't implement interfaces.

`java.lang.reflect.Method` 1.1

- `TypeVariable[] getTypeParameters()` 5.0
  gets the generic type variables if this method was declared as a generic method, or an array of length 0 otherwise.
- `Type getGenericReturnType()` 5.0
  gets the generic return type with which this method was declared.
- `Type[] getGenericParameterTypes()` 5.0
  gets the generic parameter types with which this method was declared. If the method has no parameters, an array of length 0 is returned.

java.lang.reflect.TypeVariable 5.0

- `String getName()`
  gets the name of this type variable.
- `Type[] getBounds()`
  gets the subclass bounds of this type variable, or an array of length 0 if the variable is unbounded.

java.lang.reflect.WildcardType 5.0

- `Type[] getUpperBounds()`
  gets the subclass (`extends`) bounds of this type variable, or an array of length 0 if the variable has no subclass bounds.
- `Type[] getLowerBounds()`
  gets the superclass (`super`) bounds of this type variable, or an array of length 0 if the variable has no superclass bounds.

java.lang.reflect.ParameterizedType **5.0**

- `Type getRawType()`
  gets the raw type of this parameterized type.
- `Type[] getActualTypeArguments()`
  gets the type parameters with which this parameterized type was declared.
- `Type getOwnerType()`
  gets the outer class type if this is an inner type, or `null` if this is a top-level type.

java.lang.reflect.GenericArrayType **5.0**

- `Type getGenericComponentType()`
  gets the generic component type with which this array type was declared.

You now know how to use generic classes and how to program your own generic classes and methods if the need arises. Just as importantly, you know how to decipher the generic type declarations that you may encounter in the API documentation and in error messages.

In the next chapter, you will see how the Java collections framework puts generics to work.